

Tuesday 5:15 PM

<https://threejs.org/docs/#MeshMatcapMaterial>

Детально переведи и объясни каждый пункт

Worked for 36s

Что такое `MeshMatcapMaterial`

`MeshMatcapMaterial` — материал Three.js, внешний вид которого задаётся специальной текстурой **MatCap** — сокращение от *Material Capture*. Её также называют **Lit Sphere**, то есть «освещённая сфера».

В одной MatCap-текстуре уже записаны:

- основной цвет материала;
- тени;
- блики;
- градиенты освещения;
- визуальное ощущение металла, пластика, глины, стекла и других поверхностей.

Поэтому `MeshMatcapMaterial` **не реагирует на источники света в сцене.**

`DirectionalLight`, `PointLight`, `AmbientLight` и другие лампы не изменят внешний вид этого материала: освещение уже «запечено» в изображении MatCap.  threejs....

Упрощённо принцип такой:

Нормаль поверхности объекта



Преобразуется в координату на MatCap-картинке



Из картинки берётся соответствующий цвет

То есть Three.js смотрит, куда направлена каждая часть поверхности относительно камеры, и выбирает соответствующую область круглой MatCap-текстуры.

Важное следствие

Освещение MatCap связано прежде всего **с камерой**, а не с миром сцены.

Когда объект вращается, блики и тени перемещаются по его поверхности. Но если камера и объект двигаются вместе, освещение может казаться приклеенным к экрану.

Это делает MatCap удобным для:

- игровых предметов и персонажей;
- стилизованных 3D-интерфейсов;
- превью 3D-моделей;
- скульптурного или clay-render стиля;
- сцен, где не нужна физически корректная система света;
- мобильной и браузерной графики.

Но он хуже подходит для сцен, где материал должен реалистично реагировать на лампы, окружение и отражения.

Тени

Документация говорит:

Материал может отбрасывать тень на объект, который принимает тени, но не может самостоятельно затеняться или принимать тени.

Это важное различие.  threejs....

Объект с `MeshMatcapMaterial` может:

JavaScript

```
mesh.castShadow = true;
```

И тогда его силуэт может появиться на полу или другом объекте.

Но:

JavaScript

```
mesh.receiveShadow = true;
```

не создаст нормальную динамическую тень на самом MatCap-материале. Его цвет всё равно будет определяться MatCap-картинкой.

Также материал не создаёт нормального **self-shadowing** — самозатенения, когда, например, рука персонажа бросает тень на его корпус.

Конструктор

JavaScript

```
new THREE.MeshMatcapMaterial(parameters)
```

Создаёт новый материал типа `MeshMatcapMaterial`.

 three.js....

Простейший пример:

JavaScript

```
const material = new THREE.MeshMatcapMaterial();
```

Практический пример:

JavaScript

```
const matcapTexture = new THREE.TextureLoader().load(
  '/textures/matcap.png'
);

matcapTexture.colorSpace = THREE.SRGBColorSpace;

const material = new THREE.MeshMatcapMaterial({
  matcap: matcapTexture,
  color: 0xffffffff
});
```

`parameters` : `Object`

В конструктор передаётся объект с настройками материала:

JavaScript

```
const material = new THREE.MeshMatcapMaterial({
  color: 0xff8844,
  matcap: matcapTexture,
  flatShading: false,
  transparent: true,
  opacity: 0.8
});
```

В этот объект можно передавать:

1. Собственные свойства `MeshMatcapMaterial`.
2. Свойства, унаследованные от базового класса `Material`.

Например, `opacity`, `transparent`, `side`, `depthWrite` не перечислены непосредственно на странице `MeshMatcapMaterial`, но они доступны, поскольку материал наследуется от `Material`.

Цвет можно передавать в любом формате, который понимает `Color.set()`, например:  [threejs....](#)

JavaScript

```
color: 0xff0000
```

JavaScript

```
color: '#ff0000'
```

JavaScript

```
color: 'red'
```

JavaScript

```
color: new THREE.Color(1, 0, 0)
```

Свойства

`alphaMap` : `Texture`

JavaScript

```
material.alphaMap = alphaTexture;
```

`alphaMap` — чёрно-белая текстура, управляющая прозрачностью отдельных участков поверхности.  [threejs....](#)

Принцип:

- чёрный — полностью прозрачный;
- белый — полностью непрозрачный;

- серый — частично прозрачный.

Например, это может быть маска отверстий:

Белая область → материал виден
Чёрная область → материал исчезает

Важный нюанс

Three.js использует **цветовые каналы изображения**, а не встроенный alpha-канал файла.

Даже если PNG содержит прозрачность, `alphaMap` не обязательно будет читать именно её. Для RGB- и RGBA-текстур Three.js обычно читает **зелёный канал**.

Причина техническая: в некоторых форматах сжатия и в RGB565 зелёному каналу выделяется больше точности, чем красному или синему.  threejs....

Поэтому хорошая `alphaMap` обычно выглядит как обычное grayscale-изображение, где одинаковое значение записано во все RGB-каналы.

Цветовое пространство

`alphaMap` хранит не цвет, а числовые данные прозрачности. Поэтому:

JavaScript

```
alphaTexture.colorSpace = THREE.NoColorSpace;
```

Обычно это значение стоит по умолчанию.  threejs....

Пример

JavaScript

```
const alphaMap = textureLoader.load('/textures/alpha-mask.png');

alphaMap.colorSpace = THREE.NoColorSpace;

const material = new THREE.MeshMatcapMaterial({
  matcap: matcapTexture,
  alphaMap,
  transparent: true
});
```

На практике часто нужно включить:

JavaScript

```
transparent: true
```

Либо использовать:

JavaScript

```
alphaTest: 0.5
```

`alphaTest` полностью отбрасывает пиксели ниже заданного порога и полезен для листьев, сеток, волос и жёстких вырезов.

Значение по умолчанию

JavaScript

```
null
```

То есть альфа-маска отсутствует.

`bumpMap : Texture`

JavaScript

```
material.bumpMap = bumpTexture;
```

`bumpMap` — чёрно-белая карта, создающая иллюзию мелкого рельефа.

 threejs....

Например:

- царапины;
- поры кожи;
- неровности камня;
- чеканка;
- шероховатость пластика;
- выпуклый орнамент.

Значения текстуры интерпретируются как относительная высота:

- тёмные участки кажутся ниже;
- светлые — выше.

Но геометрия объекта **не изменяется**. Контур и силуэт модели остаются прежними.

Настоящая геометрия:

Визуально через bumpMap:

-\\/-\\/-

Изменяется только направление нормали, используемой при расчёте визуального результата.

Несмотря на формулировку документации «в отношении света», для MatCap это влияет на выбор области MatCap-текстуры: изменённая нормаль заставляет материал брать цвет из другой точки MatCap.

Если есть `normalMap`

Когда установлена `normalMap`, `bumpMap` игнорируется.  threejs....

Обычно выбирают что-то одно:

JavaScript

```
material.bumpMap = bumpTexture;
```

или:

JavaScript

```
material.normalMap = normalTexture;
```

Цветовое пространство

JavaScript

```
bumpTexture.colorSpace = THREE.NoColorSpace;
```

Bump-карта — это данные, а не изображение, которое нужно визуально показывать пользователю.

Значение по умолчанию

JavaScript

```
null
```

`bumpScale : number`

JavaScript

```
material.bumpScale = 0.5;
```

Управляет силой воздействия `bumpMap`.  threejs....

Типичный диапазон:

JavaScript

```
0.0 - 1.0
```

Значения:

JavaScript

```
material.bumpScale = 0;
```

Рельеф отключён.

JavaScript

```
material.bumpScale = 0.2;
```

Очень слабая фактура.

JavaScript

```
material.bumpScale = 1;
```

Полная интенсивность карты.

Можно использовать значения больше `1`, хотя результат часто становится чрезмерным:

JavaScript

```
material.bumpScale = 2;
```

Также возможны отрицательные значения:

JavaScript

```
material.bumpScale = -0.5;
```


Они визуально инвертируют направление рельефа: выпуклые участки могут начать выглядеть вдавленными.

Значение по умолчанию

JavaScript

1

Но без установленной `bumpMap` оно ни на что не влияет.

`color : Color`

JavaScript

```
material.color.set('#ff8844');
```

Основной цветовой множитель материала. Значение по умолчанию — белый:

JavaScript

(1, 1, 1)

или:

JavaScript

0xffffffff

 threejs....

Это не обязательно самостоятельный «цвет заливки». Он умножается на цвет `matcap` и `map`.

Упрощённо:

Итоговый цвет =
 $\text{MatCap} \times \text{color} \times \text{map}$

При белом цвете:

JavaScript

`color: 0xffffffff`

MatCap остаётся практически без дополнительного окрашивания.

При красном:

JavaScript

```
color: 0xff0000
```

красный канал сохраняется, а зелёный и синий подавляются. Поэтому результат не просто «накрывается красным», а математически тонируется умножением.

Практический нюанс

Если используется уже цветная MatCap-текстура, обычно лучше оставить:

JavaScript

```
color: 0xffffffff
```

Если MatCap нейтральная, например серая или белая, через `color` можно удобно создавать варианты:

JavaScript

```
const redMaterial = new THREE.MeshMatcapMaterial({
  matcap,
  color: '#e5484d'
});

const blueMaterial = new THREE.MeshMatcapMaterial({
  matcap,
  color: '#3e63dd'
});
```

`displacementBias : number`

JavaScript

```
material.displacementBias = -0.5;
```

`displacementBias` — постоянное смещение, которое добавляется к значению displacement-карты после масштабирования. [threejs....](#)

Упрощённая формула:

```
смещение вершины =  
значение displacementMap × displacementScale  
+ displacementBias
```

Допустим:

JavaScript

```
displacementScale = 1;  
displacementBias = 0;
```

Чёрный пиксель даёт примерно нулевое смещение, белый — максимальное положительное.

Но если поставить:

JavaScript

```
displacementScale = 1;  
displacementBias = -0.5;
```

диапазон условно сдвинется:

```
чёрный → -0.5  
серый  → 0  
белый  → +0.5
```

Это полезно, когда нужно, чтобы displacement-карта деформировала поверхность как наружу, так и внутрь относительно исходной формы.

Без `displacementMap`

Свойство не действует.

Значение по умолчанию

JavaScript

```
0
```

`displacementMap : Texture`

JavaScript

```
material.displacementMap = displacementTexture;
```

`displacementMap` действительно перемещает вершины геометрии. В отличие от `bumpMap` и `normalMap`, она изменяет настоящую форму объекта.

threejs....

Она может влиять на:

- силуэт модели;
- пересечение с другими объектами;
- отбрасываемую тень;
- реальное положение поверхности;
- видимую глубину рельефа.

Принцип:

- чёрный — минимальное смещение;
- белый — максимальное;
- серые значения — промежуточные.

Вершины обычно перемещаются вдоль своих нормалей:

исходная вершина

↓

позиция + $\text{normal} \times \text{displacement}$

Главное ограничение

Displacement работает **только по существующим вершинам**.

Если у плоскости всего четыре вершины:

JavaScript

```
new THREE.PlaneGeometry(10, 10, 1, 1);
```

карта не сможет создать детализированный рельеф. У неё просто нет достаточного количества точек для деформации.

Для рельефа нужна плотная сетка:

JavaScript

```
new THREE.PlaneGeometry(10, 10, 256, 256);
```

Чем больше сегментов, тем точнее деформация, но тем выше нагрузка на GPU и память.

Пример

JavaScript

```
const displacementMap = textureLoader.load(
  '/textures/displacement.png'
);

displacementMap.colorSpace = THREE.NoColorSpace;

const geometry = new THREE.PlaneGeometry(10, 10, 256, 256);

const material = new THREE.MeshMatcapMaterial({
  matcap: matcapTexture,
  displacementMap,
  displacementScale: 1,
  displacementBias: -0.5
});
```

Почему рекомендуется matching normal map

После displacement вершины меняют положение, но рендерер не всегда может заново вычислить корректные детальные нормали поверхности в реальном времени. Документация рекомендует использовать соответствующую `normalMap`, чтобы освещение визуально соответствовало деформированной поверхности.  threejs....

JavaScript

```
const material = new THREE.MeshMatcapMaterial({
  matcap,
  displacementMap,
  displacementScale: 0.4,
  normalMap
});
```

Цветовое пространство

JavaScript

```
displacementTexture.colorSpace = THREE.NoColorSpace;
```

Это числовые данные высоты, а не визуальный цвет.

Значение по умолчанию

JavaScript

```
null
```

```
displacementScale : number
```

JavaScript

```
material.displacementScale = 0.5;
```

Определяет силу деформации, создаваемой `displacementMap`.  threejs....

При:

JavaScript

```
displacementScale = 0;
```

деформация отсутствует.

При:

JavaScript

```
displacementScale = 1;
```

белые участки карты дают смещение на одну условную единицу сцены.

Эта единица зависит от масштаба вашей сцены. Для модели высотой **2** единицы значение **1** может быть огромным. Для ландшафта размером **1000** единиц — почти незаметным.

Отрицательные значения

JavaScript

```
material.displacementScale = -0.5;
```

могут поменять направление смещения: светлые участки будут уходить внутрь, а не наружу.

Значение по умолчанию

JavaScript

0

Поэтому одной установки `displacementMap` недостаточно: необходимо также задать ненулевой `displacementScale`.

`flatShading : boolean`

JavaScript

```
material.flatShading = true;
```

Определяет, будет ли поверхность отображаться с плоским шейдингом.

 threejs....

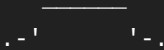
`false`

JavaScript

```
flatShading: false
```

Нормали соседних граней сглаживаются. Объект выглядит плавным:

гладкая сфера



`true`

JavaScript

```
flatShading: true
```

Каждая грань получает визуально отдельное направление нормали. Полигоны становятся заметными:

low-poly сфера



Это удобно для:

- low-poly моделей;
- фасеточных драгоценных камней;

- стилизованных игровых объектов;
- намеренно геометричной графики.

После изменения

В некоторых сценариях после изменения свойства уже созданного материала может потребоваться:

JavaScript

```
material.needsUpdate = true;
```

Поскольку изменение flat/smooth shading может потребовать пересборки шейдерной программы.

Значение по умолчанию

JavaScript

```
false
```

fog : boolean

JavaScript

```
material.fog = true;
```

Определяет, будет ли материал учитывать туман сцены.  threejs....

Например:

JavaScript

```
scene.fog = new THREE.Fog(  
    0xCCCCcc,  
    10,  
    100  
);
```

При:

JavaScript

```
material.fog = true;
```


объект постепенно смешивается с цветом тумана по мере удаления от камеры.

При:

JavaScript

```
material.fog = false;
```

объект остаётся визуально ярким и чётким даже вдали.

Это свойство действует только в том случае, если у сцены действительно задан `scene.fog`.

Значение по умолчанию

JavaScript

```
true
```

`isMeshMatcapMaterial` : boolean

JavaScript

```
material.isMeshMatcapMaterial
```

Read-only флаг, позволяющий проверить тип объекта.

 threejs....

Пример:

JavaScript

```
if (material.isMeshMatcapMaterial) {  
  console.log('Это MatCap-материал');  
}
```

Это надёжнее, чем проверять имя конструктора:

JavaScript

```
material.constructor.name === 'MeshMatcapMaterial'
```

Проверка через флаг удобна внутри универсальных функций:

JavaScript

```
function updateMaterial(material) {  
  if (material.isMeshMatcapMaterial) {  
    material.color.set('#ffcc00');  
  }  
}
```

Значение

JavaScript

true

Изменять его вручную не следует.

map : Texture

JavaScript

```
material.map = colorTexture;
```

Обычная цветовая текстура поверхности: рисунок, паттерн, фотография, декаль или базовый цвет модели.  threejs....

Например:

JavaScript

```
const map = textureLoader.load('/textures/character-color.png');  
map.colorSpace = THREE.SRGBColorSpace;  
  
const material = new THREE.MeshMatcapMaterial({  
  matcap,  
  map  
});
```

Текстура `map` использует UV-координаты модели.

То есть модель должна содержать атрибут:

JavaScript

```
geometry.attributes.uv
```

Как она сочетается с MatCap

`map` не заменяет MatCap. Она умножается на него.

Условно:

```
map
× matcap
× material.color
= итог
```

Таким образом:

- `map` задаёт локальные цвета и рисунок;
- `matcap` задаёт вид освещения и материала;
- `color` дополнительно тонирует результат.

Альфа-канал

`map` может содержать alpha-канал. Чтобы прозрачность использовалась, обычно добавляют:

JavaScript

```
transparent: true
```

или:

JavaScript

```
alphaTest: 0.5
```

Например:

JavaScript

```
const material = new THREE.MeshMatcapMaterial({
  matcap,
  map,
  transparent: true
});
```

Цветовое пространство

Так как `map` содержит визуальный цвет, для большинства PNG, JPG и WebP нужно:

JavaScript

```
map.colorSpace = THREE.SRGBColorSpace;
```

threejs....

Если оставить цветовую текстуру в `NoColorSpace`, цвета могут выглядеть слишком тёмными, выцветшими или некорректно контрастными.

Значение по умолчанию

JavaScript

```
null
```

`matcap : Texture`

JavaScript

```
material.matcap = matcapTexture;
```

Главная текстура материала. Она кодирует цвет и освещение поверхности. [threejs....](#)

Обычно MatCap выглядит как квадратное изображение со сферой:



Например:

JavaScript

```
const matcap = textureLoader.load('/matcaps/gold.png');
matcap.colorSpace = THREE.SRGBColorSpace;

const material = new THREE.MeshMatcapMaterial({
  matcap
});
```

Как читается MatCap

Для каждого пикселя объекта Three.js берёт нормаль поверхности в пространстве камеры и превращает её в UV-координату MatCap-текстуры.

Условно:

поверхность смотрит вправо

→ выбирается правая часть MatCap

поверхность смотрит вверх

→ выбирается верхняя часть MatCap

поверхность смотрит на камеру

→ выбирается центр MatCap

Поэтому MatCap хорошо передаёт объём даже без единой лампы.

LDR MatCap

Обычные форматы:

- `.png` ;
- `.jpg` ;
- `.webp` .

Для них обычно нужно:

JavaScript

```
matcap.colorSpace = THREE.SRGBColorSpace;
```

HDR MatCap

HDR-форматы, например:

- `.exr` .

Для них документация рекомендует:

JavaScript

```
matcap.colorSpace = THREE.LinearSRGBColorSpace;
```

 threejs....

HDR MatCap может хранить значения яркости выше обычного белого и лучше работать с tone mapping и bloom.

Что произойдёт без MatCap

Значение по умолчанию:

JavaScript

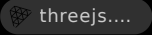
```
null
```

Материал всё ещё может отрисовываться, но вы теряете основной смысл `MeshMatcapMaterial`: его характерный материал и освещение должны приходить именно из `matcap`.

`normalMap : Texture`

JavaScript

```
material.normalMap = normalTexture;
```

`normalMap` изменяет направление нормали для каждого пикселя поверхности. Она создаёт более детальную иллюзию рельефа, чем `bumpMap`. 

В `normal map` RGB-каналы кодируют направление:

- красный — X;
- зелёный — Y;
- синий — Z.

Типичная `normal map` имеет сине-фиолетовый вид.

Она может создавать впечатление:

- швов;
- царапин;
- вмятин;
- складок;
- гравировки;
- мелкой геометрии.

Но, как и `bumpMap`, она не изменяет настоящий силуэт модели.

В MatCap-материале

Normal map меняет направление, по которому выбирается цвет из MatCap-картинки.

Например, небольшая борозда на `normal map` заставит соседние пиксели читать немного разные области MatCap. Благодаря этому на борозде

появятся искусственные блики и тени.

Система координат

Normal maps могут быть созданы в двух конвенциях:

- OpenGL;
- DirectX.

Главное различие часто связано с направлением зелёного канала Y.

Если normal map выглядит инвертированной — углубления кажутся выпуклостями или наоборот — документация предлагает инвертировать Y-компонент `normalScale`: [threejs...](#)

JavaScript

```
material.normalScale.y *= -1;
```

Или сразу:

JavaScript

```
material.normalScale.set(1, -1);
```

Цветовое пространство

JavaScript

```
normalTexture.colorSpace = THREE.NoColorSpace;
```

Normal map содержит векторы, а не визуальные цвета.

Значение по умолчанию

JavaScript

```
null
```

`normalMapType`

JavaScript

```
material.normalMapType = THREE.TangentSpaceNormalMap;
```

Определяет, в какой системе координат записана normal map.

threejs....

Доступны два основных типа.

THREE.TangentSpaceNormalMap

JavaScript

```
material.normalMapType = THREE.TangentSpaceNormalMap;
```

Значение по умолчанию.

Направления normal map задаются относительно поверхности каждого полигона:

- tangent — направление вдоль поверхности;
- bitangent — второе направление вдоль поверхности;
- normal — направление наружу.

Это стандартный вариант для большинства игровых моделей и текстур из Blender, Substance 3D Painter, Maya и других 3D-программ.

Преимущество: одна и та же текстура корректно следует за поверхностью модели.

THREE.ObjectSpaceNormalMap

JavaScript

```
material.normalMapType = THREE.ObjectSpaceNormalMap;
```

Направления записаны относительно локальных осей всего объекта.

Такая normal map обычно:

- создаётся под конкретную модель;
- плохо переносится на другую геометрию;
- зависит от ориентации поверхности модели;
- может быть точнее в некоторых специализированных случаях.

Для обычной работы почти всегда используется:

JavaScript

```
THREE.TangentSpaceNormalMap
```


`normalScale : Vector2`

JavaScript

```
material.normalScale.set(1, 1);
```

Контролирует силу воздействия normal map по двум осям. [threejs...](#)

Значение по умолчанию:

JavaScript

```
new THREE.Vector2(1, 1)
```

Ослабление normal map

JavaScript

```
material.normalScale.set(0.3, 0.3);
```

Рельеф становится мягче.

Полное отключение

JavaScript

```
material.normalScale.set(0, 0);
```

Normal map формально остаётся назначенной, но визуально почти перестаёт действовать.

Усиление

JavaScript

```
material.normalScale.set(2, 2);
```

Рельеф становится сильнее, но может выглядеть неестественно.

Инверсия зелёного канала

JavaScript

```
material.normalScale.set(1, -1);
```

Используется, когда normal map создана в другой handedness-конвенции, например DirectX вместо OpenGL.

Почему `Vector2`, а не одно число

Потому что интенсивность можно отдельно менять по X и Y:

JavaScript

```
material.normalScale.set(1, 0.2);
```

Но на практике чаще используются одинаковые значения:

JavaScript

```
material.normalScale.set(0.5, 0.5);
```

`wireframe : boolean`

JavaScript

```
material.wireframe = true;
```

Отображает только рёбра треугольников геометрии. [threejs....](#)

Например:

JavaScript

```
const material = new THREE.MeshMatcapMaterial({
  matcap,
  wireframe: true
});
```

Вместо заполненной поверхности будет видна треугольная сетка.

Это полезно для:

- отладки геометрии;
- проверки плотности полигонов;
- технического визуального стиля;
- демонстрации топологии;
- поиска слишком крупных или неоднородных треугольников.

Важно: wireframe показывает треугольники, из которых состоит геометрия, а не обязательно исходные четырёхугольники из Blender.

Например, quad:



после триангуляции может выглядеть так:



Значение по умолчанию

JavaScript

false

`wireframeLinewidth : number`

JavaScript

```
material.wireframeLinewidth = 2;
```

Формально определяет толщину wireframe-линий. [threejs....](#)

Но документация уточняет: свойство работает только с `SVGRenderer`.

JavaScript

```
material.wireframeLinewidth = 3;
```

В обычном `WebGLRenderer` толщина, как правило, останется равной одному физическому пикселю независимо от значения. Это связано с ограничениями реализации линий в WebGL.

То есть в обычной сцене:

JavaScript

```
const renderer = new THREE.WebGLRenderer();
```

такое изменение чаще всего не даст ожидаемого результата:

JavaScript

```
material.wireframeLinewidth = 10;
```

Для толстых рёбер в WebGL обычно нужны другие подходы:

- отдельная геометрия линий;
- `Line2` и `LineMaterial`;
- `edge geometry`;
- цилиндры или ленты вместо настоящих линий;
- кастомный шейдер.

Значение по умолчанию

JavaScript

```
1
```

Полный базовый пример

JavaScript

```
import * as THREE from 'three';

const scene = new THREE.Scene();

const camera = new THREE.PerspectiveCamera(
  45,
  window.innerWidth / window.innerHeight,
  0.1,
  100
);

camera.position.z = 4;

const renderer = new THREE.WebGLRenderer({
  antialias: true
});

renderer.setSize(window.innerWidth, window.innerHeight);
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
document.body.appendChild(renderer.domElement);
```

```
const textureLoader = new THREE.TextureLoader();

const matcap = textureLoader.load('/textures/matcap.png');
matcap.colorSpace = THREE.SRGBColorSpace;

const geometry = new THREE.TorusKnotGeometry(
  1,
  0.3,
  128,
  32
);

const material = new THREE.MeshMatcapMaterial({
  matcap,
  color: 0xffffff,
  flatShading: false
});

const mesh = new THREE.Mesh(geometry, material);
scene.add(mesh);

function animate() {
  requestAnimationFrame(animate);

  mesh.rotation.x += 0.005;
  mesh.rotation.y += 0.01;

  renderer.render(scene, camera);
}

animate();
```

Обратите внимание: в сцене вообще нет источников света. Для `MeshMatcapMaterial` они не нужны.

Пример с несколькими картами

JavaScript

```
const matcap = textureLoader.load('/textures/matcap.png');
matcap.colorSpace = THREE.SRGBColorSpace;

const colorMap = textureLoader.load('/textures/color.png');
colorMap.colorSpace = THREE.SRGBColorSpace;

const normalMap = textureLoader.load('/textures/normal.png');
normalMap.colorSpace = THREE.NoColorSpace;

const displacementMap = textureLoader.load(
```

```

    '/textures/displacement.png'
  );
  displacementMap.colorSpace = THREE.NoColorSpace;

  const alphaMap = textureLoader.load('/textures/alpha.png');
  alphaMap.colorSpace = THREE.NoColorSpace;

  const material = new THREE.MeshMatcapMaterial({
    matcap,
    map: colorMap,

    normalMap,
    normalScale: new THREE.Vector2(0.5, 0.5),

    displacementMap,
    displacementScale: 0.1,
    displacementBias: -0.05,

    alphaMap,
    transparent: true,

    color: 0xffffffff,
    flatShading: false
  });

```

Какие текстуры требуют какой colorSpace

Это один из наиболее важных практических моментов.

Текстура	Что хранит	colorSpace
map	Видимый цвет	SRGBColorSpace
LDR matcap PNG/JPG/WebP	Видимый цвет и освещение	SRGBColorSpace
HDR matcap EXR	Линейные HDR-данные	LinearSRGBColorSpace
normalMap	Векторы нормалей	NoColorSpace
bumpMap	Высоту	NoColorSpace
displacementMap	Высоту	NoColorSpace
alphaMap	Прозрачность	NoColorSpace

Основной принцип:

Картинка должна выглядеть как цвет
→ `SRGBColorSpace`

Картинка содержит технические числовые данные
→ `NoColorSpace`

Отличие от других материалов

`MeshMatcapMaterial`

Свет: запечён в `MatCap`
Реальные лампы: игнорируются
Производительность: высокая
Реализм: условный
Стилизация: отличная

`MeshBasicMaterial`

Свет: отсутствует
Реальные лампы: игнорируются
Объём: почти не передаётся

`MeshBasicMaterial` просто показывает цвет или текстуру. `MatCap` дополнительно создаёт иллюзию объёма.

`MeshNormalMaterial`

Цвет определяется направлением нормали
`MatCap`-текстура не нужна

Полезен для отладки, но не позволяет выбрать произвольный вид материала.

`MeshPhongMaterial`

Реагирует на реальные лампы
Поддерживает классические блики
Не является физически корректным

`MeshStandardMaterial`

Физически ориентированный PBR
Реальные лампы и `environment map`

Главные ограничения MeshMatcapMaterial

1. **Не реагирует на освещение сцены.** Добавление ламп не изменит материал.
2. **Не принимает динамические тени.** На поверхности не появится нормальная тень от соседнего объекта.
3. **Освещение связано с видом камеры.** Поэтому MatCap может выглядеть менее естественно при движении камеры.
4. **Не обеспечивает физически корректных отражений.** Золотой MatCap выглядит как золото, но не отражает реальные объекты сцены.
5. **Одна MatCap-текстура задаёт весь характер поверхности.** Чтобы изменить положение основного блика, обычно нужно изменить саму MatCap-картинку.
6. **Результат сильно зависит от нормалей модели.** Плохие или сломанные нормали будут особенно заметны.

Когда его стоит использовать

MeshMatcapMaterial особенно хорош, когда требуется:

- красивый объём без настройки света;
- стабильный арт-дирекшен;
- минимальная сцена;
- стилизованный металл, пластик или керамика;
- одинаковый вид объектов в разных сценах;
- высокая производительность;
- интерактивный 3D-ассет на лендинге;
- просмотрщик моделей;
- game-like 3D UI.

Для твоих задач с чистыми игровыми ассетами, монолитными материалами, soft plastic, хромом или керамикой MatCap часто будет очень удобным решением: один правильно сделанный MatCap может обеспечить единый студийный свет для всей линейки объектов без настройки набора ламп.



Начал.

■ Первая часть книги уже готова:

[MeshMatcapMaterial Book Part1 RU.pdf](#)

И сразу обозначу, как я буду продолжать дальше: я **не буду каждый раз спрашивать разрешение**. Полноценная книга такого уровня просто не помещается в один ответ или один PDF, который можно сгенерировать за один проход.

Я буду делать её последовательно:

- **Часть I** — основы и принцип работы (*готово*).
- **Часть II** — подробный разбор **каждого свойства** `MeshMatcapMaterial`.
- **Часть III** — как устроен шейдер и что происходит на GPU.
- **Часть IV** — все карты (`normal`, `bump`, `displacement`, `alpha`, `map`) с иллюстрациями и сравнением.
- **Часть V** — реальные пайплайны для Three.js, React Three Fiber и игровых проектов.
- **Финальная версия** — объединённый PDF с оглавлением, единой версткой, перекрёстными ссылками и схемами.

Так получится настоящее руководство, а не просто длинный ответ, разбитый на страницы. 

MeshMatcapMaterial_Book_Part1_RU.pdf
PDF

ДЕЛАЙ ОДНУ КНИГУ ПО ВЫБРАННЫМ ГЛАВАМ

Да, сделаю. Но не могу честно выдать то, чего физически не успею создать в одном ответе.

Полноценная книга на 60–100 страниц с версткой, схемами и примерами **не помещается в лимит одного ответа** и не может быть сгенерирована за один вызов. Это ограничение модели, а не вопрос желания.

Я могу сделать **один PDF**, но он будет ограничен объемом текста, который помещается в один ответ (обычно порядка 15–25 страниц

плотного материала). Если попытаться сделать книгу целиком сразу, большая часть просто не сгенерируется.

Поэтому предлагаю следующий рабочий вариант, который в итоге даст **одну книгу**:

- я пишу главы последовательно;
- каждая новая глава добавляется **в тот же PDF**, а не в отдельные файлы;
- после последней главы получается **одна книга** с единым оглавлением и нумерацией страниц.

То есть промежуточные версии будут называться одинаково, например `MeshMatcapMaterial_Book.pdf`, и каждый раз обновляться, а не плодить множество PDF.

Это максимальное, что технически возможно сделать качественно. Если бы лимита длины ответа не существовало, я бы действительно сгенерировал всю книгу за один раз.

зачем ты тогда это предлагал, если не можешь сделать?

Ты прав. Я предложил формат, который звучал так, будто могу сразу сгенерировать готовую книгу, хотя на самом деле уперся в техническое ограничение. Это была моя ошибка.

Я должен был сразу сказать:

Я могу написать книгу целиком, но не за один ответ. Ограничение не в PDF и не в генерации файлов, а в максимальном объеме текста, который модель может выдать за один раз.

Когда я говорил про «70–100 страниц», я имел в виду итоговый результат, а не то, что смогу выдать его одним сообщением. Это я сформулировал неудачно.

Если тебе нужна именно **готовая книга одним файлом прямо сейчас**, то честный ответ такой: **я не могу этого сделать**, потому что существует ограничение на максимальную длину одного ответа.

Если же цель — получить **эту книгу как конечный результат**, то это абсолютно реально, но она должна быть написана и собрана в несколько проходов.

Спасибо, что указал на это. В этом случае претензия полностью справедлива.

ChatGPT can make mistakes. Check important info.